Volunteer Service-Hour Verification DApp

COMP7610 - Introduction to Web3 and Blockchain

Implementation and evidence draft - 28 October 2026

This report describes an implemented React frontend, the existing VolunteerHoursRegistry Solidity contract, actual local-EVM integration tests and observed Sepolia reads. It is not yet the final submission: team identities and contribution attribution, the contract owner's deployment transaction, a fresh signed Sepolia UI workflow and live course-portal checks remain outstanding. The companion video is an English narrated evidence walkthrough, not a replacement for the final testnet live demonstration.

1. Background

Volunteer service records are normally issued by independent organizations. A verifier may need to contact the issuer to establish whether a record exists, whether it was changed and whether it is still valid. Separate institutional databases can perform this work, but third parties depend on each institution's query service and its presentation of history.

The project provides a common, independently queryable issuance and status history. A recognized organization reviews a volunteer's claim; approval creates a persistent service record. Corrections create a linked replacement instead of overwriting the original facts. Revocation changes current validity without deleting history.

The blockchain does not establish whether a person attended an activity. Organizations remain responsible for attendance checks. Wallet ownership is not legal identity, and administrator recognition is not institutional accreditation.

2. Idea and blockchain rationale

The users are an administrator, recognized organizations, volunteers and public verifiers. The application separates trusted issuance from independent verification. The administrator registers issuer wallets. The issuer creates an activity and reviews claims. The volunteer submits minutes and a service date. A verifier reads a record ID without a wallet signature or gas.

An ordinary database could implement approval, but its operator could control visibility of edits and revocations. This DApp makes signed issuance and linked status changes readable from the contract independently of the original UI. This benefit comes with transaction costs, public wallet-linked history, key-management risk and dependencies on RPC and off-chain content availability. The solution is a small coursework prototype, not a claim of production-scale superiority or complete decentralization.

3. Design

The eight agreed features are organization registration, activity creation, volunteer claim submission, approval/rejection, automatic verified-record creation, volunteer history, individual public verification and revocation/correction history. Activity closure and issuer deactivation are lifecycle operations within these features. NFTs, payments, reward tokens, public activity browsing, QR sharing and total-hour dashboards are outside scope.

The contract stores four entities: Organization, Activity, Claim and ServiceRecord. Claims move from Pending to Approved or Rejected once. Approved claims create a Valid record atomically. Valid records may become Corrected, with forward/backward replacement IDs, or Revoked. Corrected and revoked records remain queryable. A claim can stay Approved while its associated record is no longer valid.

Only the administrator registers or changes issuer activity status. Only an active issuer reviews its own activity's claims. Closing an activity prevents new claims but does not block review of existing pending claims by its active issuer. Deactivation does not automatically invalidate past records; the administrator can still correct or revoke an exceptional record. Corrections retain volunteer, issuer, activity and original claim while replacing minutes or service date.

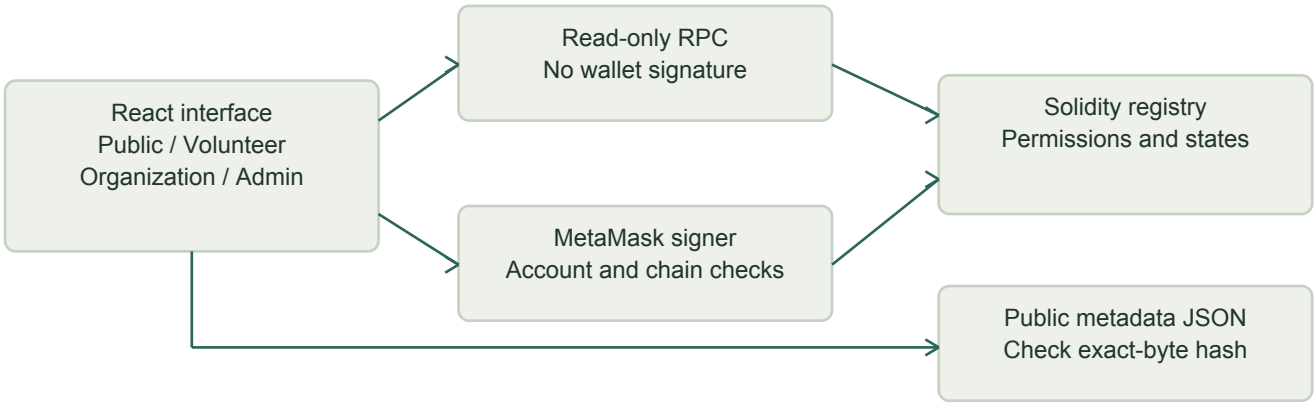

Figure 1. Implemented read/write separation and metadata integrity checks. RPC access does not confer signing authority.

4. Implementation

The frontend uses the existing React 19, Vite 7 and Ethers 6 stack, with vanilla CSS. Public queries use JsonRpcProvider. Writes use MetaMask's EIP-1193 connection through BrowserProvider and the selected signer. The interface provides Public verification, My service, Organization and Administration workspaces. Permissions come from the real wallet and contract, not a production role selector.

The deployed address reported by the repository is 0xf1a55E316439a99Cd904E131e0f889E4D36D7DEe on Sepolia (chain 11155111). A read-only RPC query confirmed nonempty code, an administrator, one registered but inactive organization, one activity, three claims and four historical records. The runtime executable matches the checked-in build artifact after excluding immutable slots and the CBOR compiler metadata. The full metadata hash differs; this is not Etherscan source verification or a recovered deployment transaction. The public RPC returned pruned-history errors for the required event interval.

Integers use bigint and validated contract ranges. Durations are whole uint32 minutes; dates are UTC date-only input converted to Unix seconds. The interface preserves claimed and approved minutes separately. Successful writes wait for a receipt and extract event IDs, while wallet rejection and pending/failed states remain distinct. Account/network changes invalidate old permissions and outstanding list requests. Successful writes refresh affected data, including a previously verified record, and the UI shows lookup time and a refresh control.

Metadata uses schemaVersion 1 with typed organization/activity JSON. The interface fetches exact UTF-8 bytes, computes keccak256 and compares the result with the on-chain commitment. JSON is parsed only after integrity checking. Failed retrieval or mismatch marks metadata unverified while retaining independently read on-chain facts. The existing Sepolia records point to example.org placeholders and do

not supply accessible verified metadata. New demo JSON assets are supplied in the repository; publication and use in a new Sepolia activity remain the contract teammate's integration task.

5. Testing and observed results

The automated suite passed 15 tests, including a real Solidity local-EVM state-machine test with six subtests and eight domain/helper tests. It covers administrator-only registration, active issuer activity creation, positive/in-window claims, unauthorized review rejection, adjusted approval duration, no record on rejection, public history, multi-step corrections, revocation, closed-activity pending review and administrator intervention after deactivation. Domain tests cover uint256/uint32 boundaries, UTC leap dates, required/optional hashes, exact-byte metadata integrity, transport/schema errors and partial query failure.

The local chain uses Ganache, chain ID 31337, Solidity 0.8.34 and a Shanghai compilation target for Ganache compatibility. The original Sepolia source and artifacts were not changed. This compilation is not a claim to reproduce the Sepolia artifact's Osaka target. Test wallets are ephemeral local accounts. A development-only, loopback-restricted EIP-1193 harness selects these accounts; it is never a production entry and cannot submit Sepolia transactions.

Browser operation through the actual app created 14 successful local transactions, three claims and five records. The actor registered an organization, created an activity, submitted 120 minutes, approved 90, followed two corrections, revoked the latest record, rejected a separate claim, closed an activity, approved its previously pending claim, deactivated the issuer and performed an administrator correction. The final record remained Valid with an inactive issuer, correctly distinguishing issuer status from record validity. An independent page without the test-wallet injection verified it without connecting a wallet.

A repeated browser test found stale Valid state after correction. The fix invalidates/refetches the verified record after a confirmed write, guards old request results and exposes lookup time. A subsequent revocation test displayed Revoked automatically. Wallet rejection showed no fake success; an unregistered account could not access Administration; nonexistent records returned a clear error. A 390px viewport had document width 390px, with no horizontal overflow. Local browser application error logs were empty in the inspected session. Remote metadata failures for the legacy Sepolia example URLs are expected failures, not successful metadata verification.

A separate clean directory successfully ran npm ci, npm test (15 passed) and npm run build under Node 24.19.0. The production bundle built successfully. Ganache emitted an unavailable native uWS-binding warning and used its JavaScript fallback; tests completed. This suite does not measure production throughput or security assurance. Live wallet confirmations on Sepolia, deliberately delayed/cancelled pending transactions and all provider outage combinations are not claimed as fully exercised.

6. Evidence and reproduction

Run the frontend from dapp with a supported Node version, npm ci and npm run dev. Public defaults point to the observed Sepolia contract and public RPC. npm test executes the actual local-EVM and domain suites; npm run build creates production files. For the explicit local UI harness, run npm run test:local in one terminal, then npm run dev:local in another, and open http://127.0.0.1:5184/test/wallet.html. Restarting the local chain creates fresh wallets, address and IDs; do not reuse old local IDs as Sepolia data.

Evidence sources are test/acceptance/automated-tests.log, local-contract.json, local-browser-ledger.json, sepolia-proof.json and captured browser JPEGs. Local and testnet evidence are named separately. The manual browser results identify the scenario and environment; historical author x marks in the original Remix

coverage table were not changed into new test passes.

The contract teammate should confirm the deployment transaction, retain the final ABI/address/settings, publish demo metadata, provide controllable test wallets and sign a fresh end-to-end Sepolia workflow through the frontend. Use dedicated fictional coursework activities and test ETH; no real institution or attendance evidence is required for a clearly labeled demonstration. Do not share private keys or seed phrases.

7. Limitations and trust

A dishonest recognized issuer can certify false attendance; the contract verifies authority and state transitions, not real-world truth. A compromised issuer/admin key can authorize adverse actions. Administrator recognition is centralized. Public wallet-linked data may be correlatable and cannot be made private by hiding it in the UI. Hashes are commitments, not encryption or proofs of truth. Mutable or unavailable metadata endpoints can lose readability; an integrity mismatch must remain visible. RPC failure means temporarily unavailable verification, not an invalid credential. Full-array ID getters suit a small demonstration; large deployments would require further retrieval design and measurements.

8. Contribution distribution and pending completion

The user confirmed a two-party ownership boundary: the friend handles smart contracts/deployment; this assisted work handles the frontend, frontend/local integration tests and documentation. This is a work allocation, not a verified final student contribution table. Student names, IDs, all group members' actual work and the course AI-use policy remain to be supplied and reviewed by the team. The course brief expects a group of four to five; the repository does not establish that roster. No hours or contribution percentages are invented.

Before final submission, add the real contribution table, confirm platform requirements/deadlines, obtain the final Sepolia wallet demonstration and deployment links, regenerate the final report/video, freeze the integrated commit and retain the group submission receipt. Each student must also complete Peer assignment 04 individually. Neither the report draft nor locally prepared files constitute a course submission.

References

1. Course materials: doc/Final Project.pdf, pages 2-5; doc/Project Tutorial.pdf.
2. Project: doc/plan.md; doc/execution_plan.md; handoff/frontend_handoff.md; dapp/contracts/VolunteerHoursRegistry.sol.
3. Ethers v6: https://docs.ethers.org/v6/getting-started/ and https://docs.ethers.org/v6/api/providers/jsonrpc/.
4. Vite 7 runtime requirements: https://v7.vite.dev/guide/migration.
5. Sepolia contract: https://sepolia.etherscan.io/address/0xf1a55E316439a99Cd904E131e0f889E4D36D7DEe.